

EXP52 / EXP53 · GS_FLOATERLAB

VIGS-SLAM 실시간성 타임라인

직렬 체인과 GS Mapping, 얼마나 겹치고 어디가 남는가

1253 시퀀스 · 65.1초 실녹화 · imu_cpp+TensorRT 가속 적용 · 2026-07-20

SUMMARY

세 가지 질문에 대한 답

RTX 5090이면 실시간이 될까?

부분적으로만. VIGS 저자 공식 벤치마크(RTX 5090)에서도 tracking만은 39.83fps(여유)지만 tracking+mapping은 12.02fps로 여전히 목표(30fps) 미달. 27.0초 오버헤드 중 21.1초(순수 Python 루프·IPC)는 GPU 세대와 거의 무관하게 남을 가능성이 큼 — GPU 업그레이드는 필요조건이지 충분조건이 아니다.

직렬 체인이 gs_mapping보다 작은가?

거의 같다 — 89.6초 vs 90.5초. 어느 한쪽만 0으로 줄여도 나머지 하나가 이미 예산(65.1초)을 넘는다. 두 축을 동시에 줄여야만 실시간에 도달한다.

27.0초 오버헤드의 정체는?

model_loading 1.5초(1회성) + queue_get_wait 4.4초(리더 프로세스 IPC 대기) + 미계측 잔여 21.1초(pbar 문자열 포맷·이미지 텐서 복사·save_trajectory의 GPU→CPU 전송+파일 I/O로 추정 — 아직 세분 계측 안 함, exp53 신규 측 후보).

순차 실행 — gs_parallel 없이 돌리면



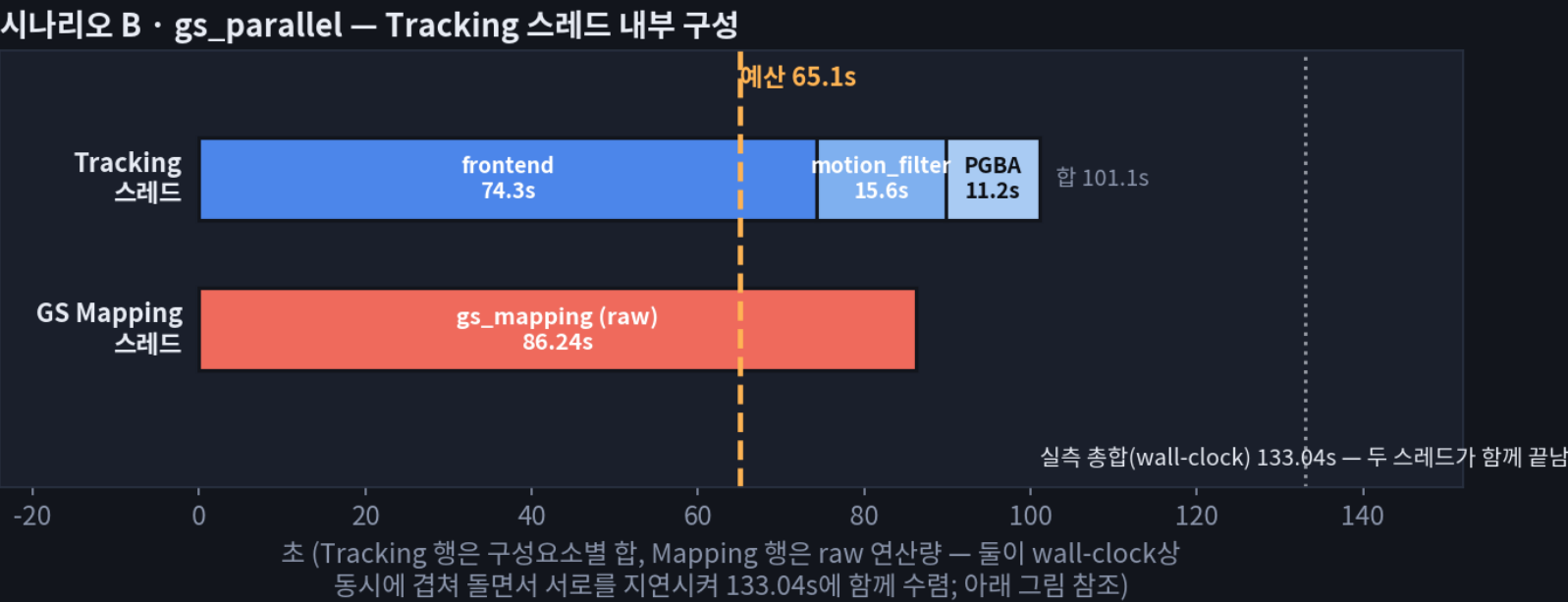
180.10s
총합

2.77×
실시간 배수

50.2%
gs_mapping 비중

한 프레임당 motion_filter→frontend→PGBA→gs_mapping이 전부 같은 스레드에서 순서대로 실행된다고 가정했을 때, 전체 시퀀스에 걸쳐 각 단계가 누적인 시간의 구성.

gs_parallel — 두 스레드가 GPU를 나눠 쓰면



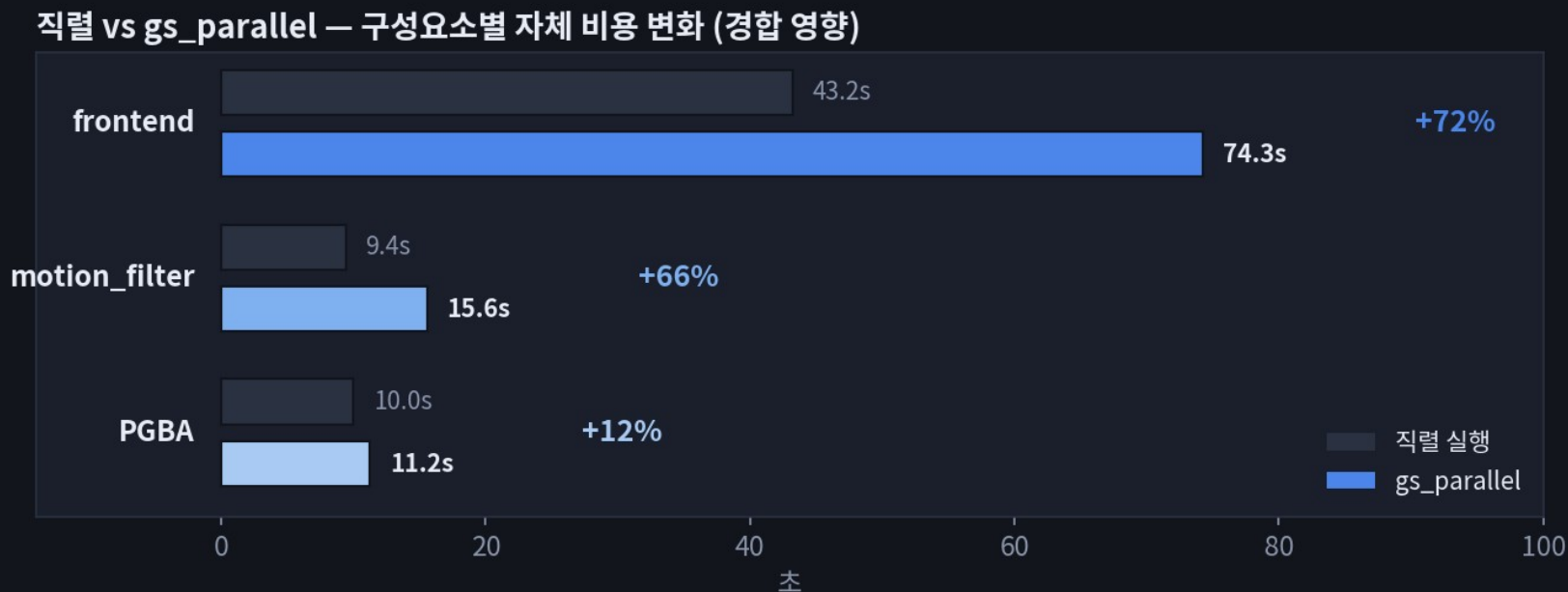
133.04s
총합(실측)

2.04×
실시간 배수

-26.1%
순차 대비

두 스레드 다 133.04초에 함께 끝난다 — 한쪽이 먼저 끝나고 기다리는 구조가 아니라, 서로가 서로를 GPU 경합으로 늦추며 같은 시점에 수렴한다.

왜 frontend가 병렬일 때 오히려 느려지나

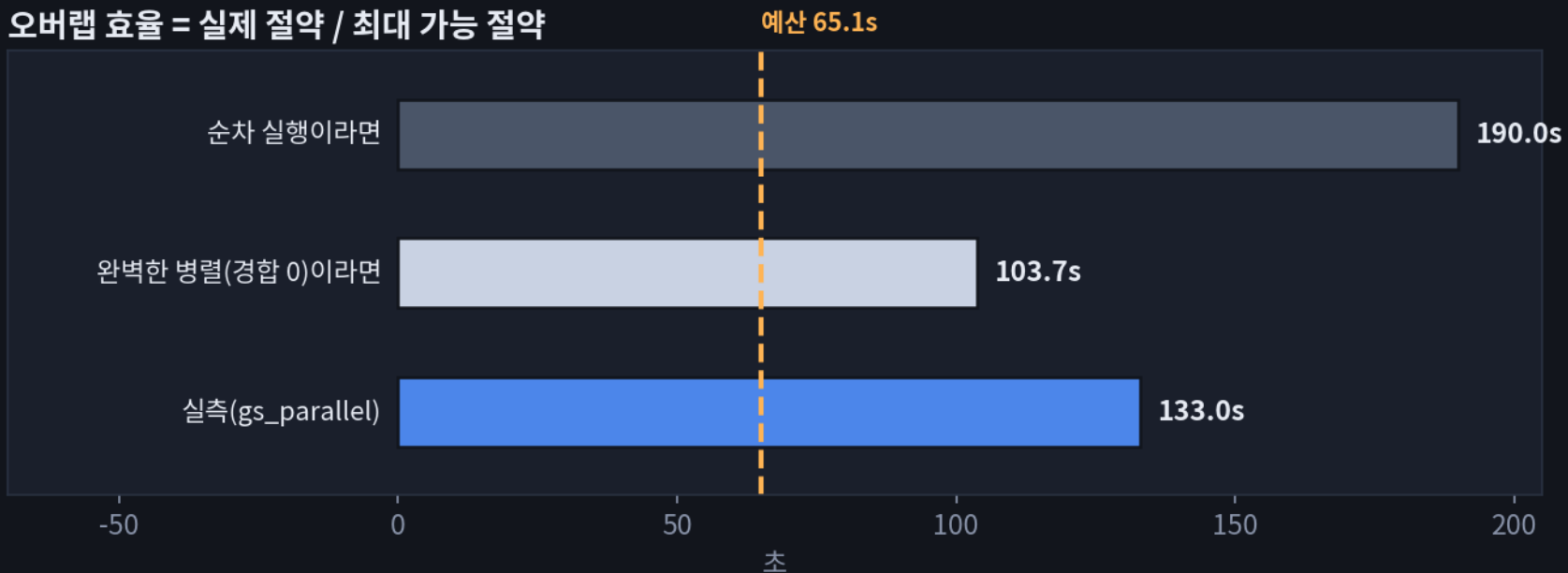


GPU 경합은 매핑 쪽에만 일어나는 게 아니라 양방향이다 — `bundle_adjust(BA solve)`와 `update_op_forward(GRU)`는 frontend에서 가장 무거운 GPU 커널인데, `rasterize/backward`처럼 `gs_mapping`이 던지는 무거운 커널과 같은 SM·메모리 대역폭을 놓고 경쟁한다.

반면 PGBA(`pgba_run`)는 sparse pose graph 최적화라 상대적으로 가벼워 거의 안 느려짐(+12%) — mapping 자체도 거의 그대로($90.5 \rightarrow 86.24s$). 가설(미검증, nsys 트레이스 필요): mapping이 던지는 크고 지속시간 긴 커널 뒤에서 frontend의 작고 빈번한 커널들이 줄서서 기다린다.

오버랩 효율 — 실제로 얼마나 겹치는가

오버랩 효율 = 실제 절약 / 최대 가능 절약



실제 절약 = $189.96 - 133.04 = 56.92s$
최대 가능 절약 = $189.96 - 103.72 = 86.24s$
오버랩 효율 = $56.92 / 86.24 = 66.0\%$ (나머지 34% = GPU 경합으로 못 숨긴 시간)

66.0%

오버랩 효율

34.0%

GPU 경합으로 손실

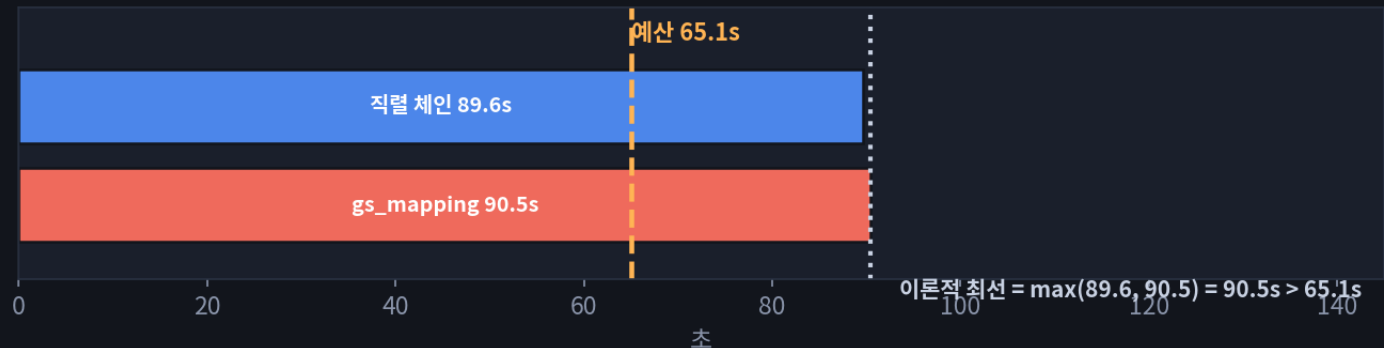
103.72s

Tracking 실측(같은 런)

※ 103.72s는 "경합이 없었을 때의 tracking 비용"이 아니라 이 gs_parallel 런에서 실측된 값 — 그 자체로 이미 위 슬라이드의 경합을 포함하고 있다.

완벽한 병렬화를 가정해도 안 되는 이유

완벽한 병렬화를 가정해도 도달 불가능한 예산



병렬화(exp52)만으로는 구조적으로 도달 불가 — 직렬 체인과 gs_mapping을 각각 65.1초 아래로 줄여야 한다.

EXP53 · 최우선

Frontend 반복 축소(iters1/iters2) — bundle_adjust+update_op_forward가 frontend 43.2s의 80%. 직렬 체인을 줄이는 가장 직접적인 레버.

EXP53

keyframe 밀도 억제(motion_filter.thresh) — keyframe 발생이 fps와 무관하다는 게 확인됐으니, 밀도 자체를 낮추는 게 fps를 낮추는 것보다 유효.

EXP53 · 신규

27.0초 오버헤드 세분 계측 — 21.1초 미계측 잔여의 정체 파악. GPU와 무관하게 남은 항목이라 별도 대응 필요.

EXP53 · 신규

축A~D를 gs_parallel 컨 상태/끈 상태 둘 다에서 측정 — frontend가 경합 시 +72% 느려짐이 확인됐으니, 순차 측정만으론 병렬 실전 효과를 과대추정할 수 있음.

EXP52 · 계속

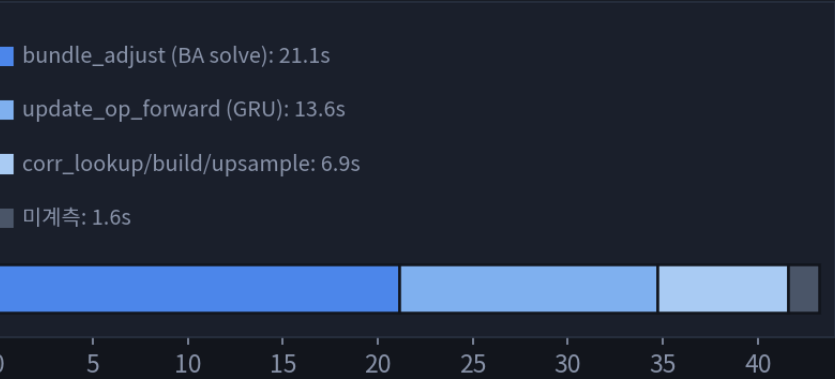
gs_mapping 자체 연산량 감소 — iteration 수·해상도·densify 빈도. RTX 5090에서도 저자 자신이 12.02fps(미달)였던 부분이라 GPU 업그레이드만으론 부족.

COMPONENT BREAKDOWN

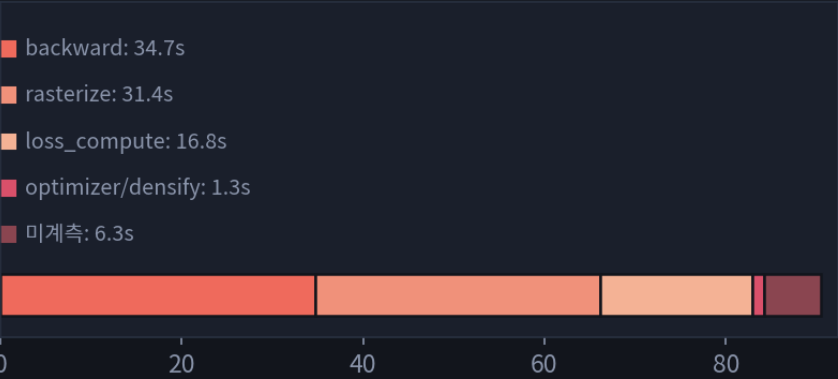
구성요소별 세부 분해

구성요소별 세부 분해 — 순차 실행 런 기준 (1253 시퀀스 누적 시간)

Frontend Tracking · 43.2s



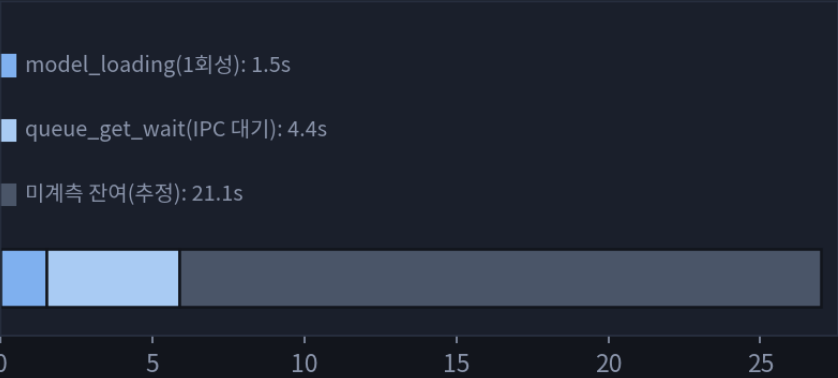
GS Mapping · 90.5s



motion_filter · 9.4s



track() 바깥 오버헤드 · 27.0s



※ gs_parallel 하에서는 frontend(43.2→74.28s) · motion_filter(9.4→15.56s)가 GPU 경합으로 더 커짐 — 뒤 슬라이드 참조